DSA & Presentation

“Good things take time”



Outline

1. DSA
    - Liveness System
2. Concurrent Programming
3. Presentation
    - Diagramming
    - Presenting

1. DSA

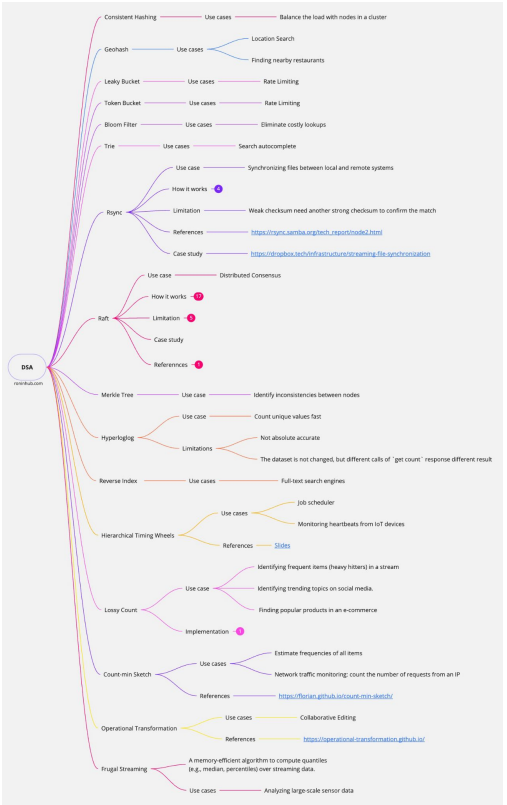
What factors do you analyze a DSA on?

Factors to Analyze a Data Structure

- Use cases
- Time Complexity
  - Read Operation
    - Random (one)
    - Range (many)
  - Write Operation: Insert, Update, Remove
- Space Complexity
  - Size
  - Location:
    - Contiguous
    - unpredictable distributed (random).
- Limitations

DSA

- Don’t need to learn or remember all.
  Put the cheat sheets into your pocket.
- Learn one when you need it
- Based on the implementation, try to find the limitations
- Note: **Don’t judge a design by the time complexity only. Consider Space Complexity, I/O operations, …**



Liveness System

Liveness System

- Context:
  - 1M IoT devices send heartbeats to server periodically (every 3 seconds).
- Functional Requirement:
  - If a device does not send any heartbeat after a timeout threshold
    → System raise alert that the device is disconnected.
  - Ability to configure the timeout threshold for each device type.
- Non-functional Requirement:
  - Scalability
  - A little delay between the timeout moment and the alert moment is acceptable

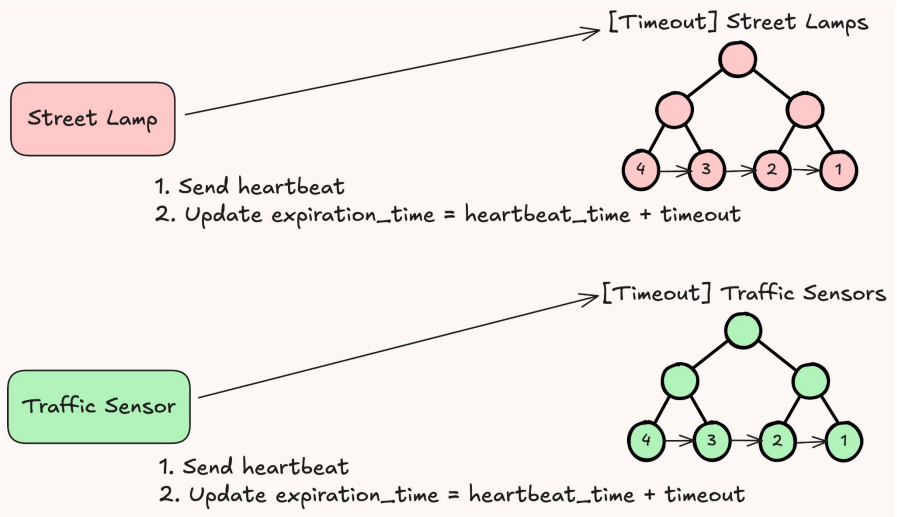
Requirement Analysis

- Every time, a device sends a heartbeat to the system, the system reset timeout timer.
  - How to reset timeout timer effectively?
- Basically, detecting timeouts is scanning the timeout moments periodically to get expired ones.
  - How to get expired ones effectively?
  - How long for a period actually?
- If a lot of timeouts at the same time (the same scanning), how to handle it effectively?

→ We need an appropriate data structure

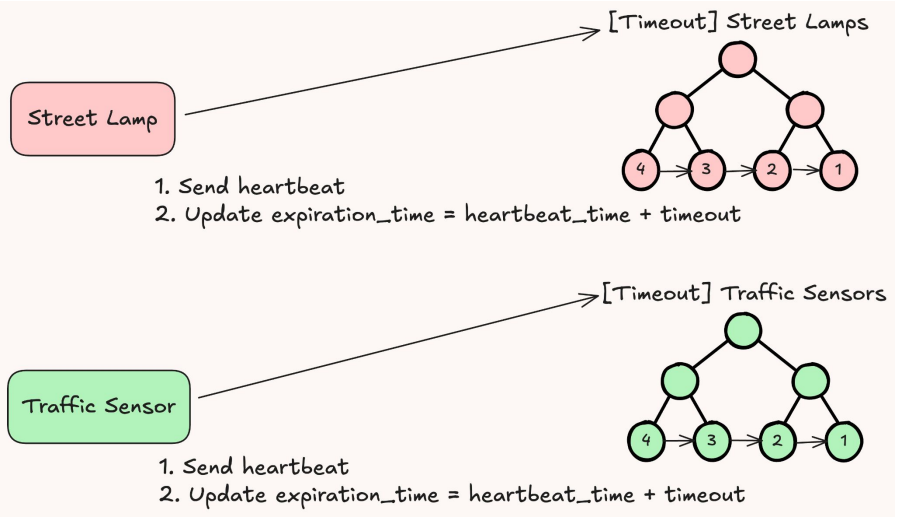
Solution 1: Sorted Set (Redis)

- Sorted Set (Redis)
  - Key: device type (for scalability)
  - Score: expiration_time
  - Member: device_id
- On heartbeat:
  - Add a new timer
  - Remove the old timer
- Advance Clock:
  - Get timers with expiration_time (score) less than current time
  - Remove this range



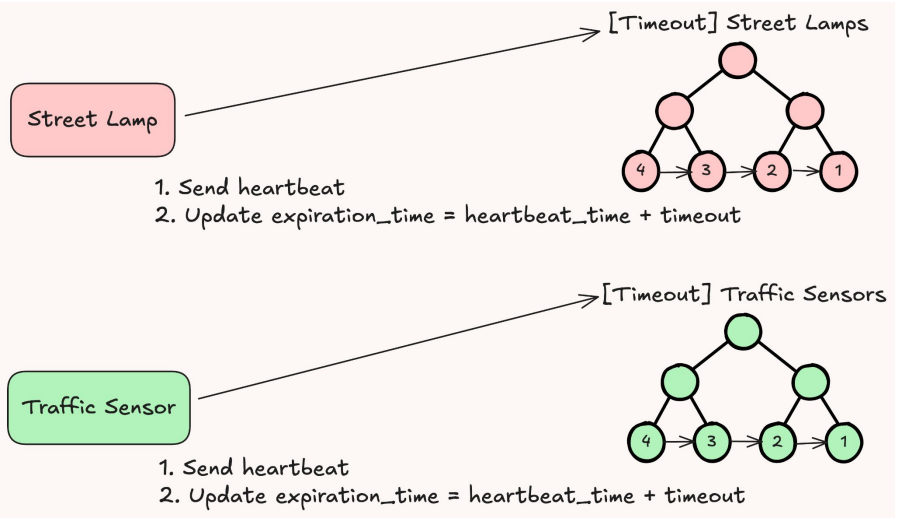
Solution 1: Sorted Set (Redis)

- Adding or removing one timer: O(log(n))
- ZRANGE…BYSCORE: O(log(N)+M)
- ZREMRANGEBYSCORE: O(log(N)+M)
    - Return the number M only, not the range
- Let's try another way: not removing the old timer



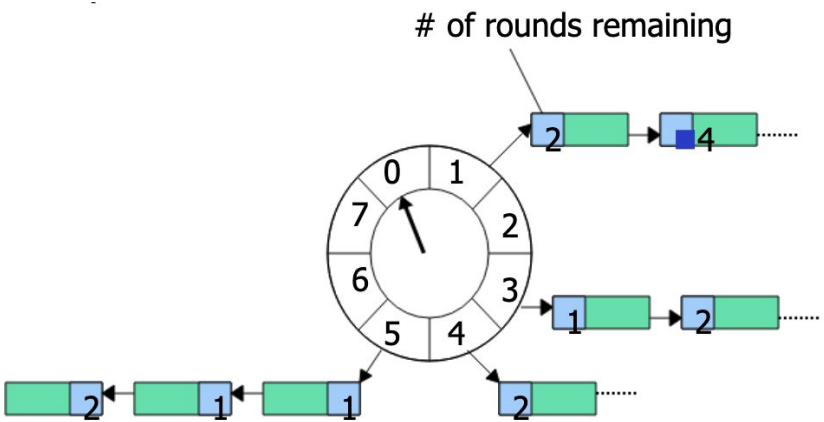
Solution 1.2: Sorted Set (Redis)

- On heartbeat:
  - Add a new timer
  - Set key:value (device_id:expiration_time)
- Advance Clock:
  - Get timers with expiration_time (score) less than current time
  - Check expiration_time
  - Remove this range
- Limitations
  - **Don’t leverage Lua script.** Because different keys
  - **More I/O calls**



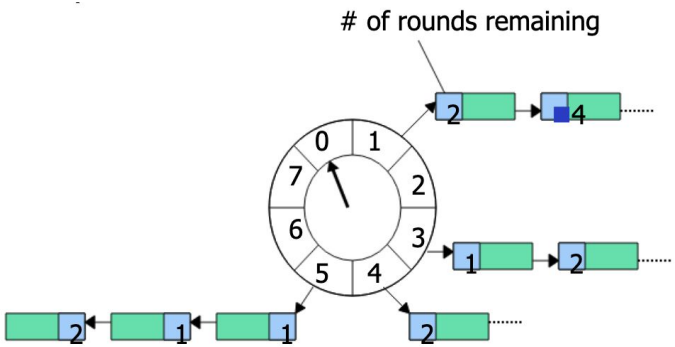
Solution 2: Timing Wheel

- Wheel: an circular array with fixed size
- The size ~ timeout threshold
- Each slot in the wheel is called a bucket. Each bucket contains a set of timers.
- Timer:
  - Device_id
  - Remaining Rounds (Expiration_time)
  - Slot Index
- Map<device_id, Timer>



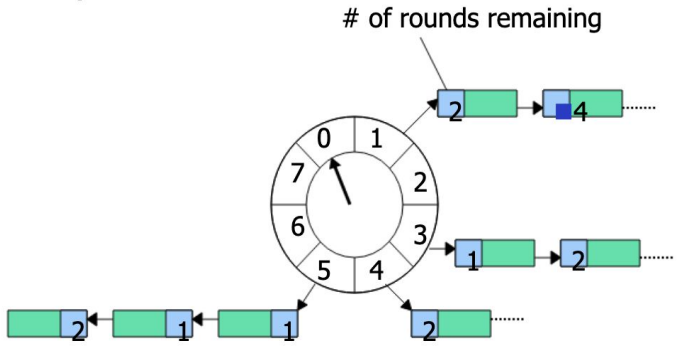
Solution 2: Timing Wheel

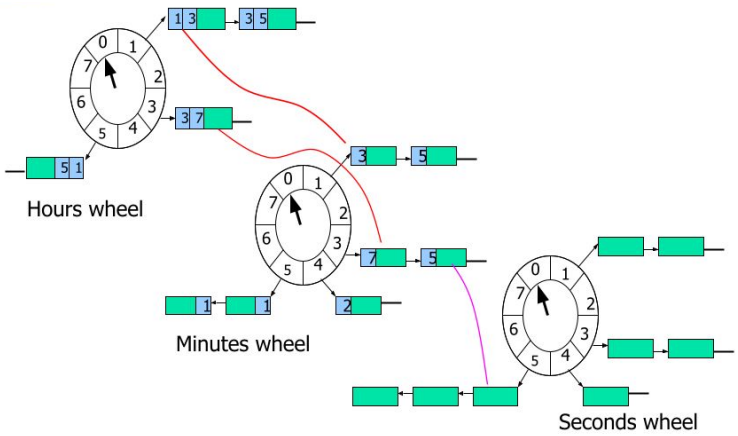
- On heartbeat:
  - Get the old timer from the map<device_id, timer>
  - Find the target bucket by slot index in timer → remove old timer
  - Calculate the new slot index → find bucket
  - Add timer into the bucket
  - Add timer into the map
  - All the above operations: O(1)



Solution 2: Timing Wheel

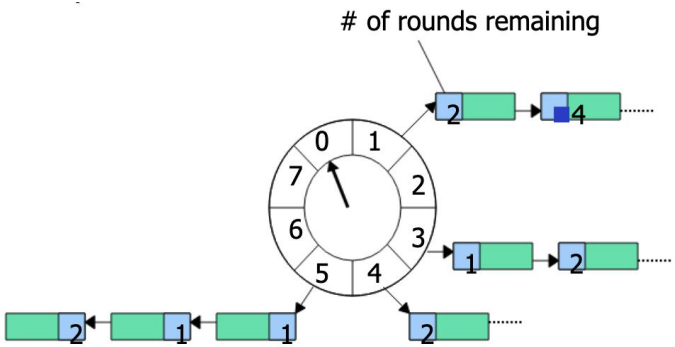
- Advance Clock: scan through a bucket
- Read more: Hashed Wheel Timer
- **Limitation: not suitable for long-duration timers**
- Kafka Hierarchical timing wheels





Problem 2: Large Bucket

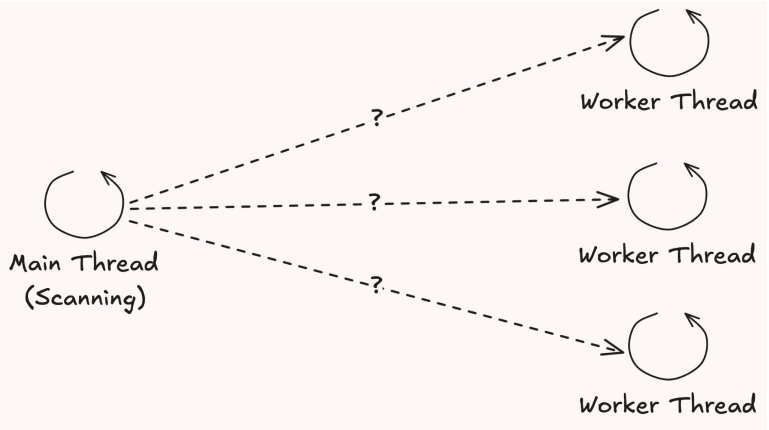
- Advance Clock: scan through a large bucket
  - Scan
  - Check the timer
  - Alert
- How to speed up the process of scanning the bucket?
- Solution: Multi-threads



2. Concurrent Programming

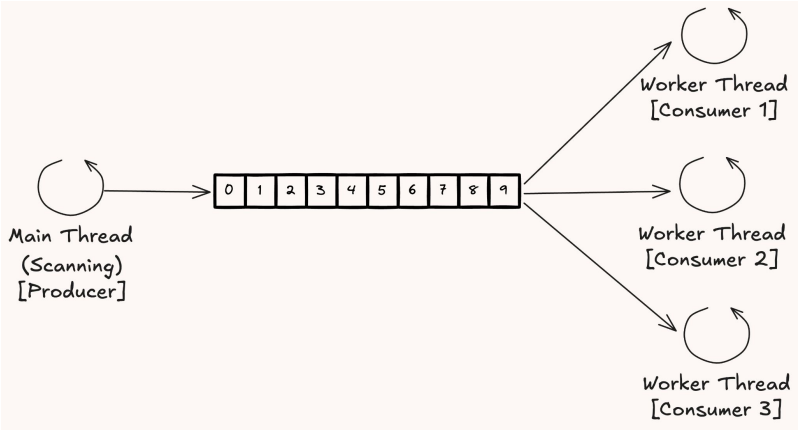
Problem 3: Messaging

- How does the main thread:
  - communicate to worker threads?
  - divide workload to worker threads?
- The sender may outpace worker threads
- Solution: Queue

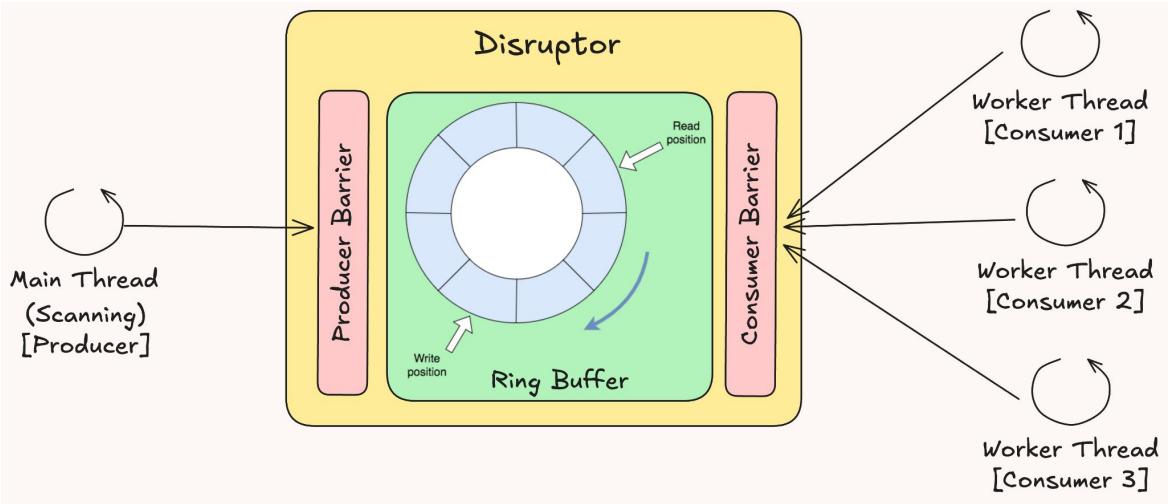


Problem 4: Queue in Concurrent Env

- Implementations
  - Array
    - Fixed size
  - Linked List
    - Unbounded → exhausting memory
    - Not cache friendly
- Write contention on the head, tail, and size variables
- Unused objects need to be reclaimed (GC)

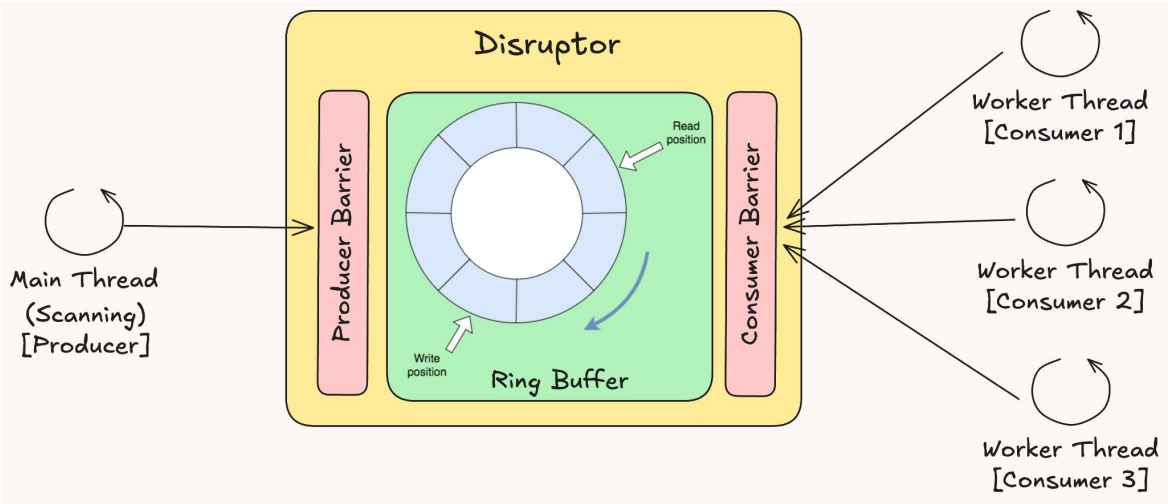


Solution 4: LMAX Disruptor



- LMAX Disruptor: High Performance Inter-Thread Messaging
- CAS instead of Lock
- Separating producer and consumer completely
- All memory for the ring buffer is pre-allocated on start up. Producers update entry, not initialize entries

Solution 4: LMAX Disruptor



- Memory Barrier to ensure visibility of changes across threads
- Batch Processing
- Highly Tunable

Solution 4: LMAX Disruptor

- Read more:
  - Learn a fundamental course of concurrent programming first.
  - LMAX Disruptor: Technical Paper
  - Dissecting the Disruptor: What's so special about a ring buffer? - Trisha Gee
  - The power of Mechanical Sympathy in Software Engineering

Table 3. Comparative throughput updated for modern hardware (in ops per sec)

	AMD EPYC 9374F – Linux 5.4.277 – OpenJDK 11.0.24		
	ABQ	**Disruptor 3**	**Disruptor 4**
Unicast: 1P – 1C	20,895,148	134,553,283	160,359,204
Pipeline: 1P – 3C	5,216,647	76,068,766	101,317,122
Sequencer: 3P – 1C	18,791,340	16,010,759	29,726,516
Multicast: 1P – 3C	2,355,379	68,157,033	70,018,204
Diamond: 1P – 3C	3,433,665	61,229,488	63,123,343

Table 4. Comparative Latency in three stage pipeline

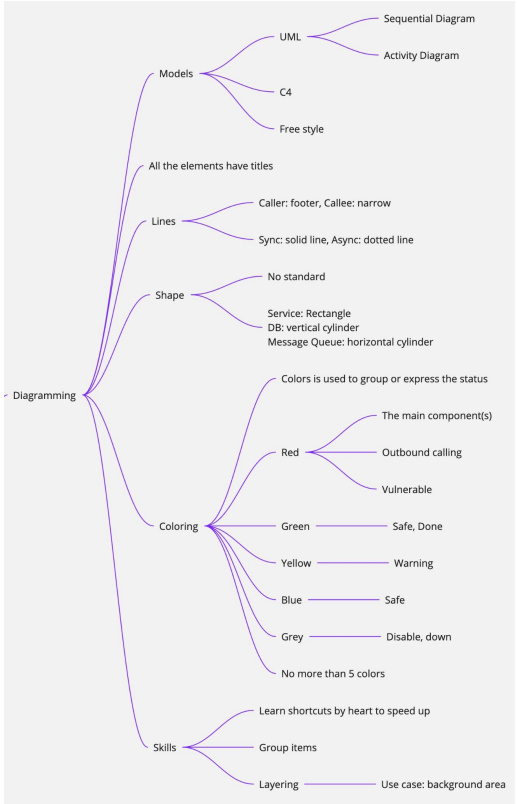
	Array Blocking Queue (ns)	**Disruptor (ns)**
Min Latency	145	29
Mean Latency	32,757	52
99% observations less than	2,097,152	128
99.99% observations less than	4,194,304	8,192
Max Latency	5,069,086	175,567

3. Presentation

3.1. Diagramming

Diagramming

- Colors have meanings
- Learn shortcuts
- Layering



3.2. Presenting

Presenting

- Top Down
- The speaker has 2 information channels: verbal and visual
- Power of Three
- Don't forget to recap



Recap

- Dont judge a design based on time complexity only.
- Disruptor is inter-thread messaging lib
- Power of Three
- 3 topics all need “Practice”!

Homework

-

References

- https://en.dtm.pub/
- https://seata.apache.org/docs/overview/what-is-seata

Thank you 🙏

